

ARIA — ENGINEERING BRIEF

The share-ready developer pack. Read top to bottom once; then use §2, §8 and §9 daily.

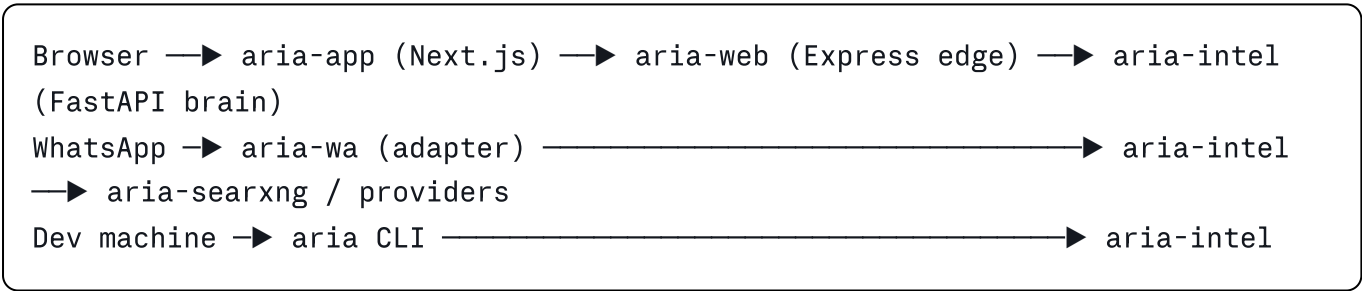
Snapshot: commit `dae009b2` · 2026-08-01 · change record R-F3623 Status vocabulary:

Implemented = evidenced by code/config in the repo · **Configured** = deploy config exists, live state not claimed · **Planned** = design only, no runtime path. Source-of-truth order when this document and reality disagree: **code + tests** → **deployment manifests** → **live probes** → **this document** → **historical docs**.

1. What you are working on (30 seconds)

ARIA is a multi-service intelligence platform: a browser UI, an identity/edge gateway, a Python intelligence brain, a WhatsApp adapter, a private metasearch service, and a local coding agent. It is a **modular monolith across a few deployable processes**, not microservices.

The one design decision that explains everything else: the Python service (`aria-intel`) is the authoritative brain. Everything else authenticates, normalizes, delivers, or renders — it does not reason, and it does not own intelligence state.



Service	Tech	Owns	Stateful
<code>aria-app</code>	Next.js 14 / React 18 / TS	Browser UI; forwards <code>/api</code> to aria-web; never calls the brain directly	No
<code>aria-web</code>	Node 22 / Express 5	Auth, sessions, roles, quotas, 2FA, proxying, billing, legacy UI	Yes
<code>aria-intel</code>	Python / FastAPI	Chat, DD workflows, RAG/knowledge, evidence, autonomy, model orchestration	Yes
<code>aria-wa</code>	Node / Baileys	WhatsApp connection + delivery only — a channel adapter, never a second brain	Yes
<code>aria-searxng</code>	SearXNG	Internal metasearch; not a system of record	Ops state
<code>aria</code> CLI	Python	Local coding agent; not in the browser path	Local

All deployed on Fly.io, region `(lhr)`, private `(*.internal)` networking.

Standing note (ADR-WA-001): the Baileys connection is designated **transitional**. Production WhatsApp architecture is ARIA's own number on the official Business Platform. Do not build new features that deepen the Baileys dependency without checking the ADR.

2. THE TWELVE INVARIANTS — the rules of the codebase

Break one of these and the PR is wrong even if it works.

1. **The Python brain is authoritative for intelligence state.** New reasoning lives in Python, not Node.
 2. **The web tier is authoritative for browser identity** and coarse role/quota enforcement.
 3. **Object ownership is re-checked at the authoritative service.** The edge is not a substitute for object-level checks.
 4. **Channel adapters never become reasoning brains.**
 5. **Every user-visible action reports success AND failure. Delivery is separate from generation** — a generated answer is not a delivered answer.
 6. **Domain code uses the shared abstractions** (state store, RAG store, model factory, brain wiring) — never raw DB connections or vendor SDKs.
 7. **Facts and reports retain provenance. Absence of evidence is never a negative finding.**
 8. **Unknown or unmeasurable state is never converted into success.**
 9. **Background work is idempotent, lease-aware, restart-safe, non-blocking.**
 10. **No new store without ownership, retention, erasure, backup, and recovery rules.**
 11. **Streaming and non-streaming paths keep behavioral parity** — any safety/audit/learning hook is applied to both.
 12. **A deployment is complete only when the live `(build_rev)` matches the intended commit.** A green image build proves nothing.
-

3. Trust boundaries (who may talk to whom)

1. **Public boundary:** browsers/public integrations terminate at `(aria-app)`, `(aria-web)`, or explicitly public FastAPI routes.
2. **Identity boundary:** `(aria-web)` owns auth, sessions, roles, quotas, and the browser→brain authorization bridge.
3. **Internal boundary:** services talk over Fly private networking with internal Bearer credentials.
4. **Intelligence boundary:** `(aria-intel)` independently validates authorization context on every object.
5. **Provider boundary:** search, LLM, email, messaging, public-data providers are **untrusted** — provenance, validation, timeouts, and failure classification are mandatory.

6. **Data boundary:** Fly volumes are per-service and single-writer; the brain volume is deliberately single-writer.
-

4. Where things live

```
crucix/
├─ aria-app/                Next.js UI
├─ aria_cli/               local coding agent
├─ aria_service/          ★ the brain
│   └─ routes/            FastAPI routers (aria.py = principal API)
│   └─ intel/             DD, RAG, evidence, state, ledger – the largest
module group
│   └─ llm/               provider factory (factory.py) – the ONLY way to
get a model
│   └─ autonomous/ guardian/ learning/ metacognitive/ personas/
│   └─ crawler/ integrations/ search_engine/ vetting/ writers/
│   └─ tests/
├─ lib/                   Node domain modules (llm, reports, search,
whatsapp, ...)
├─ services/wa-listener/   independently deployed WhatsApp adapter
├─ searxng/ public/ middleware/ scripts/ test/ docs/ data/
├─ server.mjs             Node composition root
└─ fly*.toml              deployment manifests
```

Key files for any architectural question: `(server.mjs) · (aria_service/main.py) · (aria_service/routes/aria.py) · (aria_service/aria_engine.py) · (aria_service/intel/brain_hook.py) · (aria_service/intel/state_store.py) · (aria_service/intel/rag_store.py) · (aria_service/llm/factory.py) · (services/wa-listener/aria_wa_listener.mjs) · (aria-app/lib/api.ts) · the (fly*.toml) set.`

5. The three flows you must understand

Browser chat: Browser → aria-app → aria-web (session/role/quota/ownership) → brain (Bearer + user context) → retrieval → grounded prompt → LLM → policy/evidence/quality checks → persist → respond or async job → normalized back up the chain. Sync and streaming are **separate entry paths** (invariant 11).

WhatsApp: user → aria-wa (authenticate binding, dedupe, normalize) → brain with `(request_id)` → immediate answer or async `(job_id)` → brain callback → aria-wa sends → **aria-wa reports delivery outcome + latency back to the brain.** The delivery-outcome leg is not optional (invariant 5).

Due diligence: an orchestrated multi-layer workflow (identity → corporate/ownership → sanctions/PEP → adverse info → commercial/jurisdictional → verification → synthesis → report), where every layer emits structured output + provenance, layer failures are explicit and feed completeness/confidence, reports are stored by run/case ID, and **rendering is**

downstream of structured evidence, never the data model. Read the live orchestrator + tests for the current layer set — historical docs contain stale layer counts.

6. Data layer — use the abstractions

- **State store** (`intel/state_store.py`): Redis-like async API over durable SQLite — KV, JSON, lists, hashes, sorted sets, counters, TTL, leases, scans; bounded write queue + dedicated writer; hot/cold routing; **strict read/write variants where "missing" ≠ "store failed."** Never open the databases directly.
 - **RAG store** (`intel/rag_store.py`): chunking, deterministic IDs, sanitization, content-hash dedupe, shared embeddings (baked into the image, offline), semantic + hybrid search, credibility/recency inputs, retention/rights/erasure/purge, corruption quarantine, hot→cold offload. Lives under `/data` in production.
 - **Brain wiring** (`brain_hook.absorb()` via `wire_success()` / `wire_failure()`): the single gateway through which outcomes become observable and learnable — signal recording, knowledge/RAG ingestion, learning observations, per-module health, backpressure and circuit breaking. **Every feature wires both success and failure.**
 - **Ownership:** intel owns sessions/cases/evidence/RAG/ledgers; web owns users/sessions/quotas; wa owns device credentials/bindings; app owns nothing. Cross-service copies are projections/caches with an explicit owner — never a second authoritative copy.
-

7. Models

All Python model access goes through `aria_service/llm/factory.py`. Domain modules never import vendor SDKs. The layer guarantees provider-neutral contracts, env-driven config, bounded timeouts, classified failures, cost/latency metering, retrieval-before-generation for knowledge tasks, transparent fallbacks, and response verification before success is reported. Node adapters under `lib/llm/` exist for Node-owned features only — new reasoning goes in Python (invariant 1).

8. Security rules for every PR

- Public routes are explicitly registered; everything else requires a session; privileged mutations require admin/infra roles; internal calls use the internal Bearer.
 - User-scoped identifiers are pinned/checked at the edge **and** the brain.
 - Route-specific body/upload limits at Express; document text sanitized before RAG ingestion; URLs and external content are untrusted; CLI output passes redaction.
 - Secrets live in runtime env / Fly secrets — never in the repo, browser bundles, logs, or diagrams. Internal URLs are not secrets; tokens and provider credentials are.
 - **Any new store must declare:** owner, retention class + legal basis, subject lookup/erasure, backup/recovery, provenance/correction semantics, and whether the data may enter model context or training exports (invariant 10).
-

9. Recipes (follow exactly)

New FastAPI capability: domain logic in its package (not `main.py`, not the handler) → verify signatures/sync-async → typed request/response models → **wire success AND failure before the happy path** → use state-store + factory abstractions → preserve provenance and ownership → register in the right router → explicit edge proxy route only if special auth/transformation is needed (else validate the catch-all) → tests including empty, malformed, provider-failure, store-failure.

New data source: narrow adapter, normalized result contract → record source identity, retrieval time, jurisdiction, rights, credibility tier → **distinguish no-match / unavailable / rate-limited / parse-failed / unauthorized** → wire both outcomes → only validated content enters RAG → fixtures for normal/empty/malformed/changed upstream → register health + freshness.

New channel: transport/device state in its own adapter service → normalize inbound to the brain contract with user identity + request ID → async completion for slow work → dedupe retries and callbacks → **report actual delivery outcome** → expose health, binding state, failure diagnostics.

New model provider: implement the interface under `llm/` → register in `factory.py` (no direct imports) → normalize timeout/auth/quota/safety/malformed failures → metering + provider identity in traces → test selection, success, failure, fallback through the factory.

New background loop: one-shot vs recurring → singleton/leader lease → idempotent, restart-safe, bounded shutdown → capability test that drives the real loop body → load- and latency-aware (loops share resources with interactive traffic).

10. Testing and deployment

Layer	Where
Python unit + capability	<code>aria_service/tests/</code>
CLI (tools, safety, memory, providers)	<code>aria_cli/tests/</code>
Node unit/capability (routes, proxy, security, reports)	<code>test/</code>
WhatsApp (listener, callback, delivery, watchdog)	<code>services/wa-listener/test_*.mjs</code>
Build/compile gates	scripts + package commands
Live smoke	deploy scripts/probes

Local dev: `python -m aria_service.main` · `npm start` / `npm run dev` · aria-app scripts on :3200 · `docker-compose.yml` for Node composition. URLs/credentials are env-driven — never hardcode production endpoints.

Two hard rules: changes touching `main.py` or startup deps must exercise the **lifespan path** (importing modules proves nothing — infrastructure initializes in lifespan). And a deploy is done only when live `build_rev` matches your commit — use `scripts/deploy.sh`/`deploy.ps1`, which enforce push, build revision, health checks, and live verification.

Health model: liveness (can accept traffic) $\neq$ readiness/health (dependencies working) $\neq$ diagnostic (why something is slow/stale/open-circuit) $\neq$ outcome (did the user-visible action actually complete and deliver). Don't conflate them when adding endpoints.

11. Known structural constraints (design facts, not live-health claims)

- `aria-intel` runs on a **single attached volume** $\rightarrow$ no horizontal scaling; leader/single-writer design is mandatory.
 - `server.mjs` and `main.py` are large composition roots $\rightarrow$ extractions must preserve middleware order and lifespan behavior.
 - Background work shares resources with interactive requests.
 - Browser traffic crosses two hops (aria-app $\rightarrow$ aria-web) during the frontend migration.
 - Streaming vs non-streaming drift is a real failure mode — test parity.
 - External sources vary in quality/availability — "no result" never proves absence.
-

12. PR checklist (paste into your PR description)

```
[ ] Reasoning/state changes are in aria_service (Py), not the edge or adapters
[ ] Uses state_store / rag_store / llm factory / brain wiring — no raw DB or SDK imports
[ ] wire_success AND wire_failure on every new user-visible path
[ ] Ownership checked at the brain, not just the edge
[ ] Provenance preserved; unknown  $\neq$  success; absence  $\neq$  negative finding
[ ] Streaming + non-streaming parity (if chat-path)
[ ] New store? owner/retention/erasure/backup/provenance declared
[ ] New source? no-match vs unavailable vs failed distinguished; health registered
[ ] Background loop? lease + idempotent + bounded shutdown + capability test
[ ] Tests: empty, malformed, provider-failure, store-failure paths
[ ] Lifespan path exercised if main.py/startup touched
[ ] Deployed? live build_rev verified — not just image build
```

This brief condenses `ARIA_SOFTWARE_ARCHITECTURE.md` (commit `dae009b2`). It deliberately does not report live health, live volumes, model selection, route counts, or test-pass counts; does not treat roadmap as implemented; and contains no credentials or secrets. When in doubt: code and tests win.